

Техническая документация и архитектура (Service Manual)

Обзор архитектуры

Hydra — это мульти-агентная P2P сеть, предназначенная для интеллектуальной маршрутизации трафика и обхода сетевых ограничений (DPI/цензуры). Главная инновация заключается во внедрении локальной Small Language Model (SLM) на каждом узле для динамического принятия решений о маршрутизации на основе телеметрии и контекстной диагностики.

Проект написан на **Rust** и разделен на четыре основных компонента (крейта):

1. **hydra-core** — Ядро приложения. Содержит точку входа, SOCKS5-сервер, координатора компонентов и реализацию Onion-маршрутизации (многошаговых туннелей).
2. **hydra-p2p** — Сетевой слой на базе **libp2p**. Управляет соединениями, телеметрией и протоколами связи между узлами.
3. **hydra-ai** — Модуль искусственного интеллекта. Выполняет инференс локальной LLM для выбора оптимального маршрута.
4. **hydra-econ** — Экономический слой и репутационная система (на базе **sled**).

Применяемые технологии и библиотеки

- **Сеть:** **libp2p** (TCP, Noise для шифрования, Yamux для мультиплексирования).
- **Асинхронность:** **tokio** (полный асинхронный рантайм).
- **Локальный AI:** **candle-core**, **candle-nn**, **candle-transformers** от HuggingFace (инференс GGUF моделей на CPU/GPU).
- **Модель:** **Qwen/Qwen2.5-1.5B-Instruct-GGUF** (выбрана за отличные способности к function calling и низкое потребление памяти).
- **База данных:** **sled** (встраиваемая key-value СУБД для хранения репутации пиров).
- **Форматы данных:** JSON (для промптов AI), CBOR (для бинарных сетевых протоколов диагностики).

Подробное описание компонентов

1. Сетевой слой (hydra-p2p)

Реализует P2P-сеть с использованием **libp2p**.

- **Обнаружение пиров (Discovery):** Использует Kademlia DHT для глобального поиска и mDNS для локального.
- **Туннелирование:** Использует **libp2p-stream** для создания пользовательских потоков **/hydra/tunnel/1.0.0**.
- **Телеметрия:**
 - Протокол **ping** замеряет RTT (задержку) до соседей.
 - Экспоненциальное сглаживание используется для подсчета реальной пропускной способности (Bandwidth) после каждого туннелированного соединения.

- **Co-Agent Diagnostics (Диагностика):** Кастомный протокол `request-response` (на базе CBOR). Позволяет узлу отправить запрос соседям (`DiagnosticRequest::PingTarget`) для проверки доступности ресурса (TCP Ping) с их точки зрения.

2. Искусственный интеллект (hydra-ai)

Мозг маршрутизации.

- **Подготовка промпта:** Приложение собирает список доступных пиров и их метрики (`trust_score`, `current_debt`, `rtt_ms`, `bandwidth_bps`).
- **Структурированный вывод:** Модели передается системный промпт, жестко задающий формат вывода (JSON) с массивом `path` (для Onion-маршрутизации).
- **Кэширование:** Результаты работы модели кэшируются (`moka`), чтобы избежать долгих вычислений для повторных обращений к одним и тем же хостам.
- **Диагностический контекст:** Если попытка соединения завершается ошибкой, результаты опроса соседей (Diagnostic Context) подаются в модель, и она перестраивает маршрут (самовосстановление сети).

3. Экономика и Репутация (hydra-econ)

Каждый узел ведет локальный учет:

- **Trust Score (0-100):** Повышается при успешном пропуске трафика через пира, понижается при обрывах.
- **Debt (Долг):** Учет объема переданных данных (в байтах). В будущем планируется интеграция с криптографическими settlement-решениями.
- **Хранение:** Данные персистентно хранятся на диске в директории `hydra_db`.

4. Ядро и Onion Routing (hydra-core)

- **SOCKS5 Server:** Принимает соединения от локальных приложений браузера.
- **Onion Router:** Если AI принимает решение использовать несколько прыжков (например, `Peer A -> Peer B -> Target`), ядро последовательно устанавливает stream-каналы, прося каждый промежуточный узел пробросить TCP-поток к следующему.

Архитектура сети (Как проходит пакет)

1. **User Application** отправляет запрос на `127.0.0.1:1080` (SOCKS5).
2. **hydra-core** извлекает целевой IP/Domain и порт.
3. Ядро запрашивает телеметрию по всем известным пирам из **hydra-p2p** и баланс из **hydra-econ**.
4. Формируется запрос в **hydra-ai**.
5. SLM генерирует JSON с маршрутом: `{"path": ["PeerA", "PeerB"], "transport": "vless", "max_price": 0.0}`.
6. **hydra-core** запускает **OnionRouter**, который открывает `/hydra/tunnel/1.0.0` к PeerA, передает ему команду соединиться с PeerB.
7. После успешного "рукопожатия", данные из SOCKS5-сокета проксируются через P2P stream к PeerB, который в свою очередь открывает TCP сокет к Target.

8. При ошибке на любом этапе, запускается **Co-Agent Diagnostics** — опрос других пиров о доступности Target, после чего запускается повторная генерация маршрута AI с учетом этих данных.
-

Roadmap (План дальнейшего развития)

Ближайшая перспектива (Q3-Q4 2026)

- **Обфускация трафика:** Внедрение протоколов VLESS/XTLS поверх P2P потоков для маскировки P2P-трафика от DPI под обычный HTTPS.
- **GPU Ускорение:** Добавление поддержки CUDA и Metal для **candle** при компиляции, чтобы кардинально снизить latency инференса модели на поддерживаемых системах.
- **Улучшение AI:** Переход на дообученную (Fine-Tuned) версию модели, специально натренированную на принятие решений в топологиях децентрализованных сетей.

Среднесрочная перспектива (2027)

- **Zero-Knowledge Proofs (ZKP):** Внедрение криптографических доказательств того, что пир действительно передал трафик (Proof-of-Transfer), для надежной защиты репутационной системы от накруток.
- **Децентрализованный расчет (Settlement):** Подключение блокчейн-сетей (например, Solana или TON) для взаиморасчетов микротранзакциями (миллиенты за мегабайты) на основе локального **hydra-econ** долга.

Долгосрочная перспектива

- **Полностью автономный рой:** Узлы смогут обмениваться весами моделей или LoRA-адаптерами для коллективного обучения на паттернах блокировок (Federated Learning) без передачи пользовательских данных.
- **Mesh-сети:** Расширение **hydra-p2p** для работы поверх Bluetooth LE, Wi-Fi Direct и LoRa для создания полностью оффлайновых локальных mesh-сетей.